

Dependency Injection

ليه؟!

دلوقتي هنفترض اننا عندنا برنامج البرنامج دا بيعمل اي

بيسجل ال Users الجديد ولما يسجل: بياناته تتحفظ في قاعدة البيانات و يوصله ايميل ترحيب

```
public class DataBase
{
    public void Save(string Name, String Email)
    {
        Console.WriteLine( "Saved to the database: {name} - {email}");
    }
}

public class EmailSender
{
    public void SendEmail(string Email, String Message)
    {
        Console.WriteLine( $"An email was sent to {email}: {message}");
    }
}

public class UserService
{
    public void RegisterUser(string name, string email)
    {
        // Step 1: Save to database
        var database = new Database();
        database.Save(name, email);
        // Step 2: Send an email
        var emailSender = new EmailSender();
        emailSender.Send(email, "Hello" + name + "u r welcome");
    }
    Console.WriteLine("User successfully registered");
}
```

نفهم الأول يعني إيه Dependency؟

انا عندي:

```
public class DataBase
{
    public void Save(string Name, String Email)
    {
        Console.WriteLine( $"Saved to the database: {name} - {email}");
    }
}

public class EmailSender
{
    public void SendEmail(string Email, String Message)
    {
        Console.WriteLine( $"An email was sent to {email}: {message}");
    }
}
```

و:

```
public class UserService
{
    public void RegisterUser(string name, string email)
    {
        // Step 1: Save to database
        var database = new Database();
        database.Save(name, email);
        // Step 2: Send an email
        var emailSender = new EmailSender();
        emailSender.Send(email, "Hello" + name + "u r welcome");
    }
    Console.WriteLine("User successfully registered");
}
```

```
var database = new Database();
database.Save(name, email);
```

هنا:

UserService

تعتمد على

DataBase - EmailSender

يعني `UserService` عندها Dependency على `Database` و `EmailSender`
اي كلاس ال `UserService` بتعتمد عليها فهي تعتبر dependency ليها
المشاكل اللي عندي:

هنا انا بعمل نسخه جوا الكلاس بتاعتي فتخيل مثلا انا احتجت اضيف برامتر جديد جوا ال `Database` فهنا انا لازم ارجع اغير جوا ال `UserService` وابعت لل `Database` البرامتر دا تخيل انا بق عندي كذا يوزر فهنا انا لازم اروح في كل ال `Database` اللي عندي واصيف البرامتر دا فهنا انا عندي مشكله في ال `maintenance`

مش كد وبس تخيل معايا انا عايز اشتغل على اوريكال داتا بيز بدل SQL وكل داتا بيز ليها `implementation` خاص بيها فهمنا
ال `UserService` اللي بتاخد `Database SQL` مش هينفع تاخد `Database Oracle` و انا مش هعرف احل المشكله غير اني اعمل
نسختين من ال `UserService` وكل واحده احدد فيها الداتا بيز بتاعتها وفي تكرار كود كبير جدا يعني من الاخر مفيش `flexibility`
مش كد وبس تخيل بق انا بعمل `services` لكذا حاجه مش اليوزر بس فهمنا فكل `services` انا بعمل داتا بيز مختلفه و دا بيسبب ان
scale البرنامج يكون سيئ جدا فكد `Poor scalability`

مش كد وبس انا لو عايز اعمل `test` ف انا هعمل تيست ع الداتا بيز الحقيقه اللي عندي مش بيانات وهميه فدا بردو مشكله `Hard to test`

فكد كل المشاكل دي جت بسبب ان ال `UserService` بتعمل ال `dependences` بتاعتها جواها

وهيا دي مشكله ال `Tight Coupling Problem`

اول حل ال `Constructor Injection`

```
public class UserService
{
    private Database _database;
    private EmailSender _emailSender;

    public UserService(Database database, EmailSender emailSender) //Constructor I
njection
    {
        _database = database;
        _emailSender = emailSender;
```

```
    }
    public void RegisterUser(string name, string email)
    {
        _database.Save(name, email);
        _emailSender.Send(email, "Hello " + name);
        Console.WriteLine("User successfully registered");
    }
}
class Program
{
    static void Main()
    {
        var database = new Database();
        var emailSender = new EmailSender();

        //بتاعتها جواها انا ببعتهملها من ب dependencies بتعمل ال UserService هنا بدل م ال
        //بره

        var service = new UserService(database, emailSender);
        service.RegisterUser("Ahmed", "ahmed@gmail.com");
    }
}
```

طب هنا المشاكل اتحلت ازاى

اول حاجه مشكلة ال **maintenance** هنا انا لو عدلت اى implementation كلاس بتعتمد عليها ال **UserService** ف انا مش هعدل غير ف نسخه واحده (المكان اللي بكرت فيها النسخه اللي ببعتها لل **UserService** كبرامتر بس) يعني بق فى سهوله فى ال **maintenance** وحتى لو عندي **services** لكذا حاجه ف انا خلاص مش عندي غير نسخة داتا بيز واحد ونسخه من كل حاجه فكدا مشكلة ال **scalability** اتحلت يعني عندي

الحل اسمه **Constructor Injection**, وهو ابسط شكل من اشكال ال **Dependency Injection** بدل م ال **class** تعمل ال **dependencies** بتاعتها جواها بتستلمهم من برا عن طريق ال **constructor**

بس مشكلة ال **flexibility** لسه متحلتش لان ال **UserService** بتاخد **SQLDatabase** مش حاجه تانيه حتى لو **OracleDatabase** بتعمل نفس الشغل بالظبط ال **UserService** مش هتقبلها لانها لحد دلوقتى مرتبطة باسم كلاس مش بالقدره بتاعتها احنا مش عايزين كد

احنا عايزين ال **UserService** تعتمد على **Interface** بمعنى انى هعمل **interface** للداتا بيز عموما و سواء **SQL** او **Oracle** هت **implement** ال **interface** دي وهبعتهلها وخلاص كد
ودا اسمه **Loosely Coupling (تقليل الاعتمادية):**

الكود بتاعك مش مثبت (Hardcoded) على كلاس معين. لو بكرة حبيت اضيف كلاس جديد بيستخدم **Mysql database** , كل اللي هنعمله إنتا هنكرت كلاس **MysqlDatabase** وتخليه يعمل **implement** لل **IDatabase** ونبعته لل **UserService** من غير ما نغير سطر واحد جوه ال **UserService**.

والحل دا اسمه Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules. Both should depend on abstractions.

Abstractions should not depend on details. Details should depend on abstractions.

Result:

Loose Coupling

Easy to change implementations

Easy to test

High-Level Module

هو الكلاس اللي فيه **Business Logic**.

مثال:

UserService

لأنه مسؤول عن تسجيل المستخدم.

Low-Level Module

هو الكلاسات اللي بتنفذ تفاصيل.

مثال:

```
SqlDatabase
EmailSender
```

دول بينفذوا عمليات فعلية.

قبل DIP:

```
UserService
|
+----> SqlDatabase
|
+----> EmailSender
```

بعد DIP:

```
UserService
|
+----> IDatabase
|
+----> IEmailSender
      ^
      |
      EmailSender
      ^
      |
      SqlDatabase
```

فلاستخدام الفعلي بق لو انا عندي اكثر من Services و ال services دي معتمده ع Services تانيه
وال main بقت شايلة مسؤولية انشاء كل حاجه في المشروع وكل ما المشروع كبر كل ما بقت اصعب

تخيل لو المشروع كبر وبقي عندك كذا Service، وكل Service بتعتمد على الثانية، الكود في ال `Main` هيبقى عامل كده:

```
static void Main()
{
    // محتاج داتا بيز وإيميل ولوجر OrderService، عشان أكرت ال
    var logger = new FileLogger();
    var database = new SqlDatabase(logger); // الداتا بيز بتعتمد على اللوجر
    var emailSender = new SmtplibEmailSender();

    // UserService هنا بجمعهم عشان أعمل ال
    var userService = new UserService(database, emailSender);

    // Services... وتخيل بقية ال
    var paymentService = new StripePaymentService(database, logger);
    var orderService = new OrderService(database, paymentService, emailSender, logger);

    // بقت ضخمة جداً ومليانة تفاصيل كود ملوش علاقة بوظيفة البرنامج الأساسي Main وبكده ال
    //
}
```

شايف المنظر؟ ال `Main` بقت مسؤولة عن معرفة ترتيب تكوين الكلاسات (مين يعتمد على مين ونكرت مين قبل مين). لو بكرة غيرنا ال Constructor بتاع `SqlDatabase` وخليتيه ياخد برامتر جديد، هنضطر نروحي نعدل في ال `Main`. كده إحنا منقلناش المشكلة، إحنا بس جمعناها كلها في مكان واحد (ال `Main`).

المنقذ: ال DI Container (أو ال IoC Container)

ال DI Container هو عبارة عن مكتبة أو Framework مبني جوه ASP.NET Core هو بمثابة "مخزن أو مصنع ذكي"، وظيفته مقسومة لجزأين:

1. ال Registration (التسجيل): في بداية تشغيل البرنامج، بنقول لل Container: "يا container، خد بالك، لو أي كلاس في البرنامج طلب منك `IDatabase`، روح اعمله نسخة من `SqlDatabase`".

2. ال Resolution (الطلب والتجهيز الأتوماتيكي): لما نحتاج نستخدم ال `UserService`، مش بنعمل `new` بنفسنا. بنطلب من ال Container ال `UserService` وهو أوتوماتيك بيروح يبص في ال Constructor بتاعها يلاقيها محتاجة `IDatabase` و `ISender`، فيروح يكربتهم هما الأول ويباصيهم ليها، ويديني ال `UserService` جاهزة مجهزة!

شكل ال Main بعد استخدام ال DI Container:

الكود اليدوي المعقد اللي فوق ده بيتحول لحاجة شبه كده (بمنطق ال Built-in DI في دوت نت):

```
static void Main()
{
    // 1. (المصنع) Container بننشئ ال
    var builder = WebApplication.CreateBuilder(args);

    // 2. مرحلة التسجيل (علاقات واضحة وبسيطة)
    builder.Services.AddScoped<ILogger, FileLogger>();
    builder.Services.AddScoped<IDatabase, SqlDatabase>();
    builder.Services.AddScoped<ISender, SmtEmailSender>();
    builder.Services.AddScoped<UserService>(); // سجلنا السيرفيس كمان

    var app = builder.Build();
}
```

هنا. NET Engine هو المسؤول عن الحاجات دي عن طريق ال DI Container اللي هو `builder.Services`

في Registration عندي 3 انواع من Lifetime ال `AddTransient` و `AddScoped` و `AddSingleton`

اي الفرق بينهم

عمر ال Object (Lifetime) أو "هيتعمل منه نسخة جديدة إمتى؟"

1. `Transient`

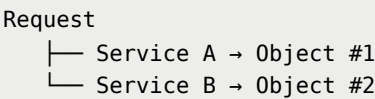
New instance every time

بيعمل Instance جديده في كل مره ال Service بيتعملها Call سواء في نفس الكلاس او في ريكويست مختلفه وده معناه ان كل Instance ليها مكان خاص بيها في الميموري

كل مرة تطلب ال Service اعمل Object جديد.

مثال:

```
builder.Services.AddTransient<IMyService, MyService>();
```



2. `Scoped`

One instance per request

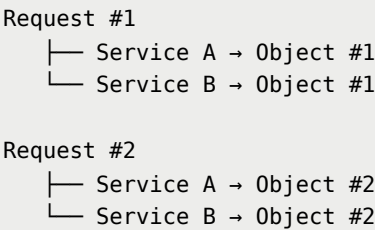
يتم إنشاء Object واحد لكل Request.

أي Service داخل نفس ال Request تستخدم نفس النسخة.

أشهر واحدة في ASP.NET Core

مثال:

```
builder.Services.AddScoped<IMyService, MyService>();
```



3. Singleton

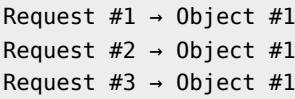
One instance for the lifetime of APP

يتم إنشاء Object واحد فقط طوال عمر التطبيق.

جميع ال Requests وال Services تستخدم نفس النسخة.

مثال:

```
builder.Services.AddSingleton<IMyService, MyService>();
```



DI Lifetimes Summary

Type	Lifetime
Transient	New Object Every Time
Scoped	One Object Per Request
Singleton	One Object For Entire Application

مثال:

```
// Registration A
builder.Services.AddTransient<IEmailService, EmailService>();

//new instance every time ->email1, email2 not Same Instance
//AreSameInstance = false


// Registration B
builder.Services.AddScoped<IEmailService, EmailService>();

//one per request ->email1, email2 in same request -> Same Instance
//AreSameInstance = true


// Registration C
builder.Services.AddSingleton<IEmailService, EmailService>();

//one for the lifetime of the app ->email1, email2 Same Instance
//AreSameInstance = true
```

```
// Controller
public class HomeController : ControllerBase
{
    private readonly IEmailService _email1;
    private readonly IEmailService _email2;
    public HomeController(
        IEmailService email1,
        IEmailService email2)
    {
        _email1 = email1;
        _email2 = email2;
    }

    [HttpGet]
    public IActionResult Test()
    {
        var same = Object.ReferenceEquals(_email1, _email2);
        return Ok(new { AreSameInstance = same });
    }
}
```